05 - 综合追问与深挖题

> 共 18 道高难度追问题。这些是面试官在你回答了基础问题后会深挖的问题，模拟真实面试中的多轮追问场景。每道题包含：面试场景 → 追问链 → 参考答案。

追问题 1: GQA 的具体实现 —— repeat_kv 怎么写?

面试场景

> 面试官：你说 MiniMind 用了 GQA，那具体实现中 KV 是怎么复制的？repeat_kv 函数怎么写?

追问链

1. GQA 的 KV 头数少于 Q 头数，计算时怎么对齐?
2. repeat_kv 的具体实现是什么?
3. 有没有不复制内存的实现方式?

参考答案

GQA 中 Q 有 8 个头，KV 只有 4 个头。每 2 个 Q 头共享 1 对 KV 头。计算注意力前，需要将 KV 从 4 头扩展到 8 头。

repeat_kv 实现:

```
def repeat_kv(x: torch.Tensor, n_rep: int) -> torch.Tensor:
    """将 KV 头复制 n_rep 次以匹配 Q 头数量
    输入: (batch, kv_heads, seq_len, d_head)
    输出: (batch, kv_heads * n_rep, seq_len, d_head)
    """
    if n_rep == 1:
        return x
    bs, n_kv_heads, seq_len, d_head = x.shape
```

```
    x = x[:, :, None, :, :].expand(bs, n_kv_heads, n_rep, seq_len, d_head)
    return x.reshape(bs, n_kv_heads * n_rep, seq_len, d_head)
```

关键点： - `expand` 不分配新内存，只是创建一个视图（view），内存高效 - `reshape` 后才会连续化内存（或者也可以用 `contiguous().view()` ） - `n_rep = q_heads / kv_heads = 8 / 4 = 2`

不复制的替代方案：可以用 `torch.repeat_interleave(x, n_rep, dim=1)` ，但 expand + reshape 更高效。也可以在注意力计算中直接用索引映射 Q 头到对应的 KV 头，完全不复制。

追问题 2: rope_theta 为什么选 1e6?

面试场景

> 面试官：你说用了 RoPE，那 rope_theta 为什么选 1e6 而不是默认的 10000?

追问链

1. rope_theta（即 base）对 RoPE 有什么影响?
2. 为什么大 base 有利于长序列?
3. 这和 NTK-aware 有什么关系?

参考答案

base 的影响:

RoPE 的频率公式为 θ_i = 1 / (base^{2i/d})。

- base 越大 → θ_i 越小 → 旋转角度变化越慢 → 相邻位置的编码差异更小
- 直觉：base 大相当于"拉伸"了位置编码的频率，让模型能区分更远的位置

为什么 1e6 有利于长序列:

原始 Transformer 用 base=10000，适合几百到几千 token 的上下文。当上下文扩展到几万甚至几十万 token 时： - base=10000 的低频分量在长序列中"绕了好几圈"，位置信息出现混叠 - base=1e6 的低频分量变化更缓慢，可以区分更远的位置 - Qwen、InternLM 等模型都选择了较大的 base（50万~100万）

与 NTK-aware 的关系:

NTK-aware 插值的核心也是调整 base：`base_new = base × scale^{d/(d-2)}`。将 base 从 10000 调整到 1e6 可以看作一种"预先做好的 NTK 调整"。这样在训练阶段就为长序列做好了准备，比训练后再做插值更自然。

追问题 3: LoRA 加在 QKV 层，为什么不加在 FFN?

面试场景

> 面试官：你说 LoRA 加在 QKV 层，那为什么不加在 FFN 层?

追问链

1. LoRA 加在注意力层 vs FFN 层有什么区别?
2. 什么时候应该加在 FFN 上?
3. 现在的最佳实践是什么?

参考答案

注意力层 vs FFN 层的功能分工： - **注意力层**：控制 token 之间的信息交互模式（"关注什么"） - **FFN 层**：对每个 token 做非线性变换，存储知识（"知道什么"）

为什么通常加在注意力层？ 1. **LoRA 原论文实验**：在注意力层的 Q/V 上加 LoRA 效果最好 2. **任务适配主要靠注意力**：SFT/对齐任务主要是调整模型的"关注模式"和"回复风格"，而非注入新知识 3. **FFN 参数量更大**：加 LoRA 的参数量也更多，效率降低

什么时候应该加在 FFN 上？ 1. 需要注入领域知识时（如医疗、法律术语） 2. 数据充足，可以支撑更多可训练参数 3. 只加注意力层效果不够好时

现在的最佳实践： - **轻量微调**：只加 Q/V（原论文配置） - **标准微调**：加 Q/K/V/O（注意力全部投影） - **全面微调**：加所有线性层（注意力 + FFN），类似 QLoRA 的做法 - 很多实践表明，对所有线性层加 LoRA（用更小的 r）比只对部分层加 LoRA（用更大的 r）效果更好

追问题 4: SFT 只计算 assistant 的 loss，具体怎么实现？loss_mask 怎么构建？

面试场景

> 面试官：你说 SFT 只计算 assistant 的 loss，那具体代码里是怎么实现的？loss_mask 怎么构建？

追问链

1. 给一条具体的对话数据，loss_mask 长什么样？
2. 多轮对话怎么处理？
3. 如果对话中有 tool_call 和 tool_response，怎么处理 mask？

参考答案

具体实现步骤：

假设一条单轮对话经 Tokenizer 编码后：

```
tokens: [BOS, sys_1, sys_2, ..., user_1, user_2, ..., ast_1, ast_2, ..., EOS]
         |-------- system --------|---- user ----|---- assistant -------|
mask:   [  0,    0,    0,  ...,    0,     0,  ...,   1,    1,   ...,  1  ]
```

代码实现：

```
def build_loss_mask(input_ids, tokenizer):
    loss_mask = torch.zeros_like(input_ids, dtype=torch.float)

   # 找到 assistant 起始标记的位置
    ast_start_token = tokenizer.encode("<|im_start|>assistant\n")
    ast_end_token = tokenizer.encode("<|im_end|>")

    in_assistant = False
    for i, token_id in enumerate(input_ids):
        if is_match(input_ids, i, ast_start_token):
            in_assistant = True
            continue
        if in_assistant and is_match(input_ids, i, ast_end_token):
            loss_mask[i] = 1  # EOS 也计算 loss
            in_assistant = False
```

```
            continue
        if in_assistant:
            loss_mask[i] = 1

    return loss_mask
```

多轮对话：每轮的 assistant 部分都标记为 1。

Tool Call 场景： - `<tool_call>` 内容是 assistant 生成的 → mask=1 - `<tool_response>` 是系统注入的 → mask=0 - tool_response 后的 assistant 回复 → mask=1

追问题 5: 梯度累积时梯度存在哪里？和增大 batch_size 有什么区别？

面试场景

> 面试官：你说用了梯度累积，那梯度存在哪里？和直接增大 batch_size 有什么区别？

追问链

1. 梯度累积的梯度存在哪里？
2. 为什么不直接增大 batch_size？
3. 梯度累积和增大 batch_size 在数学上完全等价吗？

参考答案

梯度存在哪里？

PyTorch 中，每个 `nn.Parameter` 都有一个 `.grad` 属性。`loss.backward()` 会将梯度**累加**到 `.grad` 上（不是覆盖）。只有调用 `optimizer.zero_grad()` 才会清零。

```
# 梯度存储在每个参数的 .grad 属性中
for name, param in model.named_parameters():
    print(param.grad.shape)  # 与 param 形状相同
```

所以梯度累积的"累积"就是利用了 `.backward()` 默认累加的行为。

为什么不直接增大 batch_size？

直接增大 batch_size 需要更多显存来存储： - 输入数据 + 标签 - 前向传播的中间激活值（随 batch 线性增长） - KV 缓存

梯度累积只增加了少量开销（梯度本身和 loss 计算），中间激活值是分步释放的。

数学上完全等价吗?

几乎等价，但有细微差异： 1. **BatchNorm**：累积期间各 micro-batch 的统计量不同。但 Transformer 用 LayerNorm 不受影响。 2. **Dropout**：每个 micro-batch 的 dropout mask 不同，但这属于随机性，不算本质区别。 3. **数值精度**：多次小 loss 求和 vs 一次大 batch 求均值，浮点数运算顺序不同可能有微小差异。

对于 Transformer + LayerNorm + AdamW 的组合，梯度累积与增大 batch_size 实际上可以认为是等价的。

追问题 6: DPO 的 β 设多大？太大或太小会怎样?

面试场景

> 面试官：你说 DPO 的 β 控制偏离参考策略的程度，那 β 通常设多大？太大或太小会怎样?

追问链

1. β 的常用取值?
2. β 太大/太小的影响?
3. β 和 KL 散度有什么关系?

参考答案

β 的常用取值：β ∈ [0.1, 0.5]，常见默认值 β = 0.1。

β 太大（如 β > 1）： - 模型非常保守，几乎不偏离参考策略 - 偏好学习效果很弱，模型行为接近参考模型 - 训练后几乎没有变化

β 太小（如 β < 0.01）： - 模型可以大幅偏离参考策略 - 容易过拟合偏好数据 - 可能出现"reward hacking"：模型找到提高隐式奖励但降低实际质量的捷径 - 生成质量可能下降（忘记了预训练学到的能力）

β 和 KL 散度的关系：

DPO 推导自 KL 约束的 RLHF 目标：

```
max E[r(x,y)] - β · KL(π_θ || π_ref)
```

β 就是 KL 惩罚的系数。β 越大，KL 惩罚越重，策略越不敢偏离参考模型。

实践建议： - 数据质量高、标注一致时可以用较小的 β（让模型多学习偏好） - 数据有噪声时用较大的 β（防止学到错误偏好） - 通常从 β=0.1 开始，观察训练过程中隐式奖励的差值（chosen - rejected 的 margin）来调整

追问题 7: MoE 的负载均衡问题具体是什么？怎么解决？

面试场景

> 面试官：你说 MoE 有负载均衡问题，具体是什么？怎么解决？

追问链

1. 专家坍缩是怎么发生的？
2. 负载均衡 loss 的公式和直觉？
3. 还有其他解决方案吗？

参考答案

专家坍缩的发生过程：

1. 训练初期，各专家随机初始化，性能相近
2. 某些专家偶然处理了更多 token → 获得更多梯度更新 → 性能更好
3. Router 发现这些专家更好 → 给它们分配更多 token
4. 马太效应：强者恒强，弱者越来越弱
5. 最终：1-2 个专家处理几乎所有 token，其余专家成为"死专家"
6. 模型退化为相当于只有 1-2 个 FFN 的普通 Transformer

负载均衡 Loss：

```
L_aux = α · N · Σ_{i=1}^N f_i · P_i
```

- f_i = 第 i 个专家实际处理的 token 比例（实际负载）
- P_i = Router 分配给第 i 个专家的平均概率（软概率）
- N = 专家数量
- α = 0.01~0.1

直觉：f_i 和 P_i 都大说明这个专家被过度使用，二者乘积大使 loss 增大，优化器会降低这个专家的路由概率。理想状态下每个 f_i = P_i = 1/N。

其他解决方案： 1. **Expert Choice**（反向路由）：不是让 token 选专家，而是让每个专家选固定数量的 token，天然均衡 2. **Aux-Free Load Balancing**（DeepSeek-V3）：不用辅助 loss，而是给每个专家一个可学习的 bias 来平衡路由 3. **Random Routing**：Top-2 中第二个专家按概率随机选择，增加探索性 4. **Capacity Factor**：限制每个专家能处理的最大 token 数

追问题 8: 6400 词表会不会遇到 OOV 问题？BPE 如何处理未登录词？

面试场景

> 面试官：模型只有 6400 词表会不会遇到 OOV 问题？BPE 如何处理未登录词？

追问链

1. 6400 的词表是不是太小了？
2. BPE 如何处理从未见过的词？
3. 小词表有什么优劣？

参考答案

BPE 不会有 OOV 问题：

BPE 的初始词表包含所有字节（Byte-level BPE）或所有基础字符。任何未见过的词都可以回退到字节/字符级别来表示。

例如："transformer" 如果不在词表中，可能被拆分为 "trans" + "for" + "m" + "er"，如果 "trans" 也不在词表中，继续拆分为 "t" + "r" + "a" + "n" + "s"。最极端情况下，任何文本都可以用 256 个字节来表示。

6400 词表的影响：

劣势： - 序列更长：同样的文本需要更多 token。大词表 "transformer" 是 1 个 token，小词表可能是 3-4 个 token - 序列长 → 计算量增大（Attention 的 $O(n^2)$ 复杂度） - 长距离依赖更难学习

优势： - Embedding 参数少：6400×768=4.9M vs 100000×768=76.8M - 对于 64M 参数的小模型，省下的 72M 参数很重要 - 训练更快（Embedding 查找更快） - 每个 token 出现频率更高，学习更充分

结论：6400 对 MiniMind 这个量级的模型是合理的折中。工业级模型（如 Qwen 的 15 万词表）需要大词表来提高效率，但也需要大模型来"撑起"巨大的 Embedding 层。

追问题 9: Cosine Scheduler 为什么优于 Linear？Warmup 多少步？

面试场景

> 面试官：训练用了 cosine scheduler，为什么不用 linear？warmup 多少步？

追问链

1. Cosine 和 Linear decay 的区别？
2. Warmup 步数怎么确定？
3. 有没有更好的 scheduler？

参考答案

Cosine vs Linear：

```
Cosine: lr = lr_min + 0.5 * (lr_max - lr_min) * (1 + cos(π * t / T))
Linear: lr = lr_max * (1 - t / T)
```

特点	Cosine	Linear
初期衰减	缓慢	匀速
中期衰减	快速	匀速
后期衰减	缓慢（精细调整）	匀速（可能过快）
效果	通常更好	还行但不最优

Cosine 的优势在于：训练初期保持较大学习率快速收敛，后期学习率缓慢降低做精细调整，整体收敛曲线更平滑。

Warmup 步数确定： - 通常设为总步数的 1-5% - 例如总训练 10000 步 → warmup 100-500 步 - 经验法则：小模型用较短的 warmup（100-300 步），大模型用较长的 warmup（1000-2000 步） - 也可以按 epoch 设：warmup 0.5-1 个 epoch

更好的 Scheduler： - **WSD（Warmup-Stable-Decay）**：warmup → 恒定学习率 → 最后阶段快速衰减。MiniCPM 等模型使用 - **Cosine with Restarts**：周期性重启，可能有助于跳出局部最优 - **实践中**：Cosine + Warmup 是最安全、最常用的选择，除非有特殊需求

追问题 10: 权重共享时 Embedding 和 lm_head 的梯度怎么回传?

面试场景

> 面试官：你说权重共享，那 embedding 和 lm_head 的梯度怎么回传?

追问链

1. 共享权重意味着什么?
2. 梯度怎么计算?
3. 会不会有冲突?

参考答案

共享权重的实现：

```
self.embed_tokens = nn.Embedding(vocab_size, d_model)
self.lm_head = nn.Linear(d_model, vocab_size, bias=False)
# 共享权重
self.lm_head.weight = self.embed_tokens.weight  # 同一个 tensor
```

梯度回传：

由于 `self.lm_head.weight` 和 `self.embed_tokens.weight` 指向同一个 tensor，PyTorch 自动图（autograd）会计算两条路径的梯度并**累加**：

```
∂L/∂W = ∂L/∂W(via lm_head) + ∂L/∂W(via embedding)
```

- lm_head 的梯度：来自 cross_entropy loss 对 logits 的梯度
- embedding 的梯度：来自模型前向传播对输入 embedding 的梯度

两者自动累加在同一个 `.grad` 张量上。

会不会有冲突?

理论上这两个梯度方向可能不完全一致： - lm_head 希望权重能区分不同 token 的输出 - embedding 希望权重能表达 token 的输入语义 - 但实践中这两个目标是高度一致的（语义相似的 token 在输入和输出空间都应该接近） - 共享实际上起到了正则化作用，让输入输出空间对齐 - 在小模型中（如 MiniMind），共享节省的参数量（4.9M）远大于可能的表示损失

追问题 11: 手写 GQA 的 forward 伪代码

面试场景

> 面试官：如果让你手写 GQA 的 forward 伪代码，你怎么写?

追问链

1. 从输入到输出的完整流程
2. 注意 GQA 的 KV 复制
3. RoPE 加在哪里?

参考答案

```
def gqa_forward(x, wq, wk, wv, wo, kv_cache=None, freqs_cis=None):
    """
    x: (batch, seq_len, d_model)
    wq: (d_model, q_heads * d_head)   # 768 -> 768
    wk: (d_model, kv_heads * d_head)  # 768 -> 384
    wv: (d_model, kv_heads * d_head)  # 768 -> 384
    wo: (q_heads * d_head, d_model)   # 768 -> 768
    """
    B, L, D = x.shape

    # 1. 线性投影
    q = x @ wq  # (B, L, q_heads * d_head)
    k = x @ wk  # (B, L, kv_heads * d_head)
    v = x @ wv  # (B, L, kv_heads * d_head)

    # 2. 重塑为多头形式
    q = q.view(B, L, q_heads, d_head).transpose(1, 2)   # (B, q_heads, L,
d_head)
    k = k.view(B, L, kv_heads, d_head).transpose(1, 2)  # (B, kv_heads, L,
d_head)
    v = v.view(B, L, kv_heads, d_head).transpose(1, 2)  # (B, kv_heads, L,
d_head)

    # 3. 应用 RoPE（只对 Q 和 K，不对 V）
    q = apply_rope(q, freqs_cis)
    k = apply_rope(k, freqs_cis)

    # 4. 更新 KV-Cache（推理时）
    if kv_cache is not None:
        k = torch.cat([kv_cache[0], k], dim=2)
        v = torch.cat([kv_cache[1], v], dim=2)
        kv_cache = (k, v)

    # 5. GQA 核心：复制 KV 头以匹配 Q 头数量
    n_rep = q_heads // kv_heads  # = 2
    k = repeat_kv(k, n_rep)  # (B, q_heads, L_kv, d_head)
    v = repeat_kv(v, n_rep)  # (B, q_heads, L_kv, d_head)

    # 6. 计算注意力
    scores = (q @ k.transpose(-2, -1)) / sqrt(d_head)  # (B, q_heads, L,
L_kv)
    scores = scores + causal_mask  # 应用因果掩码
    attn = softmax(scores, dim=-1)
    output = attn @ v  # (B, q_heads, L, d_head)

    # 7. 拼接多头并输出投影
    output = output.transpose(1, 2).contiguous().view(B, L, -1)  # (B, L,
d_model)
    output = output @ wo
```

```
    return output, kv_cache
```

关键点： - RoPE 只对 Q 和 K 施加，不对 V - repeat_kv 在注意力计算前，将 KV 头复制到与 Q 头数量一致 - KV-Cache 在 repeat_kv 之前更新（缓存的是未复制的 KV）

追问题 12: 从 MiniMind 做一个医疗问答模型，你会怎么做?

面试场景

> 面试官：如果让你从 MiniMind 做一个医疗问答模型，你会怎么做?

追问链

1. 整体方案设计
2. 数据从哪里来?
3. 怎么评估效果?
4. 有什么风险?

参考答案

整体方案（分阶段）：

阶段 1：继续预训练（Domain Adaptation） - 收集医疗领域文本：医学教材、论文摘要、临床指南、医学百科 - 在 MiniMind 基础上继续预训练，让模型学习医学知识 - 学习率设为预训练的 1/5~1/10 - 关键：词表可能不够。6400 词表对医学术语覆盖不足，可以考虑扩展词表

阶段 2：医疗 SFT - 构造医疗问答数据：患者提问 → 医生回答 - 数据来源： - 公开医疗问答数据集（cMedQA、MedDialog 等） - 用 GPT-4 生成高质量医疗问答对 - 人工标注的临床问答 - 使用与预训练相同的 chat_template 进行微调

阶段 3：偏好对齐 - 构造偏好数据：同一个问题给出准确回答（chosen）和不准确/有害回答（rejected） - 用 DPO 进行对齐，确保模型倾向于给出安全、准确的回答

评估方法： - 医疗知识基准测试（如 MedQA、PubMedQA） - 人工评估：请医生评估回答质量 - 安全性测试：确保不会给出有害建议

风险与应对： - **幻觉问题**：64M 模型容量有限，容易产生看似正确但实际错误的回答。应加 disclaimer。 - **安全风险**：医疗建议可能造成实际伤害。必须标注"仅供参考，请咨询专业医生"。 - **合规问题**：医疗 AI 可能涉及监管合规 - **模型容量**：64M 参数可能无法存储足够的医学知识。可以考虑 RAG（检索增强生成）来补充。

追问题 13: MiniMind 的主要局限性是什么？你觉得怎么改进？

面试场景

> 面试官：MiniMind 的主要局限性是什么？你觉得怎么改进？

追问链

1. 技术上的局限
2. 工程上的局限
3. 如果给你更多资源你会怎么做

参考答案

技术局限：

1. **模型容量有限（64M）**：
2. 无法存储大量世界知识
3. 复杂推理能力弱

4. 改进：增加模型规模到 500M-1B，或使用 MoE 扩展容量

5. **词表过小（6400）**：

6. 中文覆盖不充分，英文更差
7. 序列过长导致效率低

8. 改进：扩展到 32K-64K 词表

9. **训练数据量有限**：

10. 小规模数据可能导致知识覆盖不全

11. 改进：增加高质量训练数据，特别是多领域数据

12. **上下文长度限制**：

13. 虽然有 YaRN 支持，但小模型的长上下文建模能力仍然弱
14. 改进：增加训练时的上下文长度

工程局限：

1. **单卡训练**：虽然是优势（低成本），但限制了模型规模
2. **没有大规模评估**：在主流 benchmark 上分数不高
3. **纯教学项目**：缺少生产级的推理优化、服务化部署

如果给更多资源的改进方向： 1. 扩大模型到 1B-3B 参数 2. 扩大词表到 64K 3. 增加预训练数据到 100B+ token 4. 加入更多模态（视觉） 5. 实现更完善的评估流程

追问题 14: 预训练数据如果有毒性数据怎么办?

面试场景

> 面试官：预训练数据如果有毒性数据怎么办?

追问链

1. 什么是毒性数据?
2. 怎么检测和过滤?
3. 如果模型已经学了毒性内容怎么办?

参考答案

毒性数据的类型： - 仇恨言论、歧视性内容 - 暴力、色情内容 - 虚假信息、阴谋论 - 个人隐私信息（PII：姓名、电话、地址） - 恶意指令（如何制作危险物品等）

预训练前的过滤：

1. **关键词过滤**：维护敏感词表，包含敏感词的文本直接丢弃或打标
2. **分类器过滤**：训练一个毒性分类模型（如基于 BERT 的分类器），对每篇文档打分
3. **URL 黑名单**：过滤来自已知有害网站的内容
4. **PII 检测**：用正则表达式或 NER 模型检测并删除个人信息
5. **去重**：重复的有毒内容会被放大，去重能减轻影响

如果模型已经学了毒性内容：

1. **SFT 阶段**：通过高质量的 SFT 数据"覆盖"有毒行为，教模型拒绝不当请求
2. **RLHF/DPO 对齐**：将有毒回复标记为 rejected，安全回复标记为 chosen
3. **Safety Training**：专门添加安全相关的训练数据（如拒绝有害请求的对话）
4. **Red Teaming**：对抗测试，发现模型的安全漏洞并修补

根本认识：完全避免模型学到毒性内容几乎不可能，关键是通过对齐训练让模型学会"知道但不输出"。

追问题 15: 如果给你更多 GPU，你会怎么改进训练?

面试场景

> 面试官：如果给你更多 GPU，你会怎么改进训练?

追问链

1. 多卡训练的策略
2. 模型规模怎么扩展
3. 训练效率优化

参考答案

假设从 1×3090 → 8×A100（80GB）：

模型规模扩展： - 从 64M 扩展到 1B-3B 参数 - d_model: 768 → 2048-3072 - n_layers: 8 → 24-32 - q_heads: 8 → 16-32 - vocab_size: 6400 → 64000

训练策略升级：

1. **DDP（数据并行）**：
2. 8 卡 DDP，等效 batch_size 扩大 8 倍
3. 线性缩放学习率：lr = base_lr × 8

4. 减少梯度累积步数

5. **如果模型更大，需要模型并行**：

6. Tensor Parallelism（TP=4 或 8）：单层切分到多卡
7. FSDP（Fully Sharded Data Parallelism）：优化器状态和参数分片
8. **训练效率优化**：
9. FlashAttention 2：A100 支持良好
10. BF16 混合精度：A100 对 BF16 有原生加速
11. Gradient Checkpointing：用时间换空间，支持更大 batch
12. **数据并行增加**：
13. 增加预训练数据量到 50B-100B token
14. 更大 batch_size → 训练更稳定
15. 可以尝试更激进的学习率
16. **更完善的训练流程**：
17. 增加评估 pipeline（每 N 步评估验证集）
18. 更完善的 checkpoint 策略
19. 实验多组超参（学习率、warmup 步数等）

Scaling Law 指导： - 根据 Chinchilla 法则，1B 模型应该用约 20B token 训练 - 计算预算 $C \propto 6ND$（N=参数量，D=数据量）

追问题 16: 如果 loss 降不下去了，你会怎么排查?

面试场景

> 面试官：假设预训练跑了一段时间，loss 停在 4.0 左右降不下去了，你会怎么排查?

追问链

1. 从哪些角度排查?
2. 具体的排查步骤?
3. 常见的坑有哪些?

参考答案

系统排查清单：

1. 学习率问题： - 检查当前学习率是否太小（已衰减到极小值） - 检查 warmup 是否正确完成 - 尝试提高学习率看 loss 是否有变化

2. 数据问题： - 检查数据是否正确加载（打印几条样本验证） - 检查 Tokenizer 编解码是否正确 - 检查数据是否有大量重复 - 检查数据 shuffle 是否正常

3. 模型问题： - 检查参数初始化是否合理 - 检查梯度是否正常（有无 NaN、是否过小） - 检查各层的激活值范围 - 检查权重共享是否正确配置

4. 训练配置问题： - Causal Mask 是否正确应用 - Loss 计算是否正确（是否对 padding 也算了 loss） - 梯度累积的 loss 缩放是否正确 - 混合精度是否导致精度问题

5. 容量瓶颈： - 64M 模型在某些数据集上 loss 确实会有下限 - PPL = exp(4.0) ≈ 55，对于小模型来说可能就是接近极限了 - 对比 loss 曲线：如果平滑下降然后 plateau，可能就是模型容量限制

快速诊断步骤：

```
1. 打印 lr：确认学习率调度正常
2. 打印梯度范数：确认梯度在合理范围
3. 检查一个 batch 的数据：确认数据正确
4. 用小模型跑小数据看能否过拟合（如果不能，说明模型或训练有 bug）
5. 关闭 AMP 纯 FP32 训练排除精度问题
```

追问题 17: 多轮对话中的注意力是怎么工作的？历史轮次的信息是怎么利用的？

面试场景

> 面试官：多轮对话中，模型是怎么利用前面几轮的信息的？

追问链

1. 多轮对话拼接后注意力怎么计算？
2. 历史轮次的信息会衰减吗？
3. 怎么处理上下文过长的情况？

参考答案

多轮对话的注意力机制：

多轮对话被拼接为一个长序列：

```
[System][User1][Assistant1][User2][Assistant2][User3][...]
```

在 Causal Mask 下，当模型生成 Assistant3 的回复时，它可以注意到序列中所有之前的 token（System、User1、Assistant1、User2、Assistant2、User3）。注意力机制允许模型从任意历史位置提取信息。

历史信息会衰减吗？

理论上不会：Self-Attention 可以从任意远的位置直接关注。但实际上： 1. **注意力稀释**：序列越长，注意力权重分散到更多位置，每个位置获得的注意力减少 2. **RoPE 的距离衰减**：RoPE 编码的相对距离越远，注意力分数倾向于越低 3. **模型容量限制**：小模型难以充分利用很长的上下文

上下文过长的处理： 1. **截断**：只保留最近 N 轮对话（从头部截断，保留最新内容） 2. **摘要压缩**：用模型将历史对话总结为简短摘要 3. **滑动窗口**：只保留最近 window_size 个 token 的完整注意力 4. **增大上下文窗口**：通过 YaRN 等技术扩展支持的上下文长度

追问题 18: 为什么 MiniMind 不用 FlashAttention？什么时候该用？

面试场景

> 面试官：MiniMind 用的是标准注意力实现，为什么不用 FlashAttention？

追问链

1. MiniMind 不用 FlashAttention 的原因
2. 什么规模应该考虑 FlashAttention？
3. FlashAttention 有没有限制？

参考答案

MiniMind 不用 FlashAttention 的原因：

1. **教学目的**：MiniMind 是学习项目，标准实现更透明，方便理解注意力机制的每一步
2. **规模小**：64M 参数、序列长度通常不超过 512-1024，标准注意力的 n^2 开销可以接受
3. **依赖简洁**：FlashAttention 需要编译 CUDA kernel，增加了环境依赖
4. **兼容性**：纯 PyTorch 实现可以在更多设备上运行

什么时候该用 FlashAttention? - 序列长度 > 2048 时，标准注意力的内存和速度瓶颈开始显现 - 需要节省显存来增大 batch_size 时 - 生产环境中需要优化推理速度时 - 使用 A100/H100 等支持良好的 GPU 时

FlashAttention 的限制： 1. 需要 CUDA 编译，CPU 和某些 GPU 不支持 2. 不支持所有类型的注意力 mask（如自定义的稀疏 mask） 3. 不方便查看中间的注意力权重（debugging 困难） 4. 某些 head_dim 不支持（通常限制为 64/128） 5. FlashAttention 3 已有较好的泛用性，但仍限 NVIDIA GPU

实践建议： - 学习/实验阶段：用标准实现 - 训练阶段：PyTorch 2.0+ 的 `F.scaled_dot_product_attention` 可自动选择 FlashAttention（如果可用） - 生产部署：必须用 FlashAttention 或类似优化